66.7
Pass

Akanksha Srivastava

Other

PDF generated at: 26 Feb 2026 09:22:20 UTC

View this report on HackerRank

Score

66.7% • 150.0 / 225

scored in eClerx - FO Java, Unix & Python Assessment in 87 min 49 sec on 26 Feb 2026 13:22:24 IST

Candidate Information

Email	akankshasrivastava3108@gmail.com
Test	eClerx - FO Java, Unix & Python Assessment
Candidate Packet	View
Taken on	26 Feb 2026 13:22:24 IST
Time taken	87 min 49 sec/ 90 min
Work Experience	> 5 years
Invited by	Vipul
Invited on	24 Feb 2026 18:09:33 IST

Skill Distribution

No.	Skill	Score
1	Bash Intermediate	0%

2	Java Intermediate	100%
3	Python Intermediate	100%

Tags Distribution

Bash	0%	Sorting	0%
Medium	66%	Java	100%
Multithreading	100%	Interfaces	100%
OOP	100%	Context Manager	100%
Python	100%	Classes	100%

Questions

Linux/Unix • 0 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Bash: Maximum Size File Coding	Bash (Intermediate)	0/75	-	51 min 46 sec	⊗

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
2	Java: Safe Cracking Coding	Java (Intermediate)	75/75	-	23 min 26 sec	✓

Python (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
3	DB Context Coding	Python (Intermediate)	75/75	-	11 min 28 sec	✓

1. Bash: Maximum Size File

Incorrect

Coding Bash Sorting Medium

Question description

Given the output of the Unix `ls -l` command, which includes details like file sizes for multiple files, write a script that identifies and prints the name of the file with the largest size.

Example

Consider the output of the `ls -l` command as follows:

```
-rw-r--r-- 1 user group   1024 Aug  1 12:00 file1.txt
-rw-r--r-- 1 user group   2048 Aug  1 12:00 file2.txt
-rw-r--r-- 1 user group    512 Aug  1 12:00 file3.txt
```

The largest file in this output is file2.txt, so the result should be printed as follows:

```
file2.txt
```

Function Description

Complete the function *find_max_size_file* in the editor with the following parameters:

string ls_output: a multi-line string containing the output of the Linux `ls -l` command

Outputs

string: the name of the largest file

Constraints

- The total number of lines can be up to 1,000.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of lines in *ls_output*.

Each of the next *n* lines contains a line from *ls_output*.

▼ SAMPLE CASE 0

Sample Input 0

```
5
-rw-r--r--  1 user  staff   1234 Jul 30 14:22 file1.txt
drwxr-xr-x  3 user  staff    102 Jul 31 09:15 Documents
-rwxr-xr-x  1 user  staff  20480 Jul 29 21:05 script.sh
-rw-r--r--  1 user  staff  40960 Jul 28 10:30 image.png
drwxr-xr-x  5 user  staff    170 Jul 30 18:42 Projects
```

Sample Output 0

```
image.png
```

Explanation

The input lists 5 files with their details. The output is image.png, which is the file with the largest size (40960 bytes)

▼ SAMPLE CASE 1

Sample Input 1

```
8
-rw-r--r--  1 user group  4096 Jul  1 12:34 file1.txt
drwxr-xr-x  2 user group  4096 Jun 28 09:21 directory1
-rw-r--r--  1 user group  2048 Jun 30 18:47 file2.txt
```

```
drwxr-xr-x  3 user group  4096 Jul  2 07:50 directory2
-rwxr-xr-x  1 user group  8192 Jul  3 11:11 script.sh
-rw-r--r--  1 user group 10240 Jul  4 15:32 report.pdf
drwxr-xr-x  4 user group  4096 Jul  5 08:19 project
-rw-r--r--  1 user group  5120 Jul  6 13:45 image.png
```

Sample Output 1

```
report.pdf
```

Explanation

The input lists 8 files with their details. The output is report.pdf, which is the file with the largest size (10240 bytes)

Interviewer guidelines

▼ SOLUTION

Bash Solution

```
find_max_size_file() {
   local ls_output=$1
   echo "$ls_output" | sort -k 5 -n | tail -n 1 | awk '{print $9}'
}
```

Candidate's Solution

Language used: **BASH**

```
#!/usr/bin/env bash

#
# Complete the 'find_max_size_file' function below.
#
# The function is expected to return a STRING.
# The function accepts following parameters:
#  1. STRING ls_output
#

find_max_size_file() {
    local ls_output=$1

    # Write your code here
```

```
    local max_size= -1
    local max_file=""

    while read -r line; do
        if [-n "$line"]; then

            set -- $line
            current_size =$5
            current_file =$9

                if [ "$current_size" -gt "$max_size"]]; then
                        max_size = $current_size
                        max_file=$current_file
                fi

        fi

        done <<EOF
    $ls_output
EOF


        echo "$max_file"





}

main() {
    local n_lines
    read -r n_lines
    local ls_output=""

    for ((i = 0; i < n_lines; i++)); do
        read -r line
        ls_output+="$line"$'\n'
    done
    # echo "$ls_output"
    max_size_file=$(find_max_size_file "$ls_output")
    echo "$max_size_file"
}
main
exit 0
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Wrong Answer	0	0.0065 sec	1.63 KB
TestCase 1	Easy	Sample	Wrong Answer	0	0.0083 sec	1.75 KB
TestCase 2	Easy	Sample	Wrong Answer	0	0.0085 sec	1.63 KB
TestCase 3	Easy	Hidden	Wrong Answer	0	0.0094 sec	1.75 KB
TestCase 4	Easy	Hidden	Wrong Answer	0	0.0087 sec	1.63 KB
TestCase 5	Easy	Hidden	Wrong Answer	0	0.0115 sec	1.63 KB
TestCase 6	Medium	Hidden	Wrong Answer	0	0.0492 sec	1.88 KB
TestCase 7	Medium	Hidden	Wrong Answer	0	0.0542 sec	1.75 KB
TestCase 8	Medium	Hidden	Wrong Answer	0	0.0546 sec	1.75 KB

TestCase 9	Hard	Hidden	Wrong Answer	0	0.5022 sec	1.75 KB
TestCase 10	Hard	Hidden	Wrong Answer	0	0.5134 sec	1.88 KB
TestCase 11	Hard	Hidden	Wrong Answer	0	0.5027 sec	1.88 KB
TestCase 12	Hard	Hidden	Wrong Answer	0	0.5187 sec	1.88 KB

No comments.

2. Java: Safe Cracking

Correct

Coding Medium Java Multithreading Interfaces OOP

Question description

Implement a *SafeCracking* class that executes a sequence of operations to rob a bank vault. The class should coordinate with an *IActions* interface to perform the robbery in the correct order.

Your implementation must follow a specific sequence of operations:

1. First, unlock the vault door.
2. Then, simultaneously determine the safe deposit box number and hack its PIN.
3. Finally, open the safe deposit box using the obtained information.

Method to Implement

```
void rob(IActions actions)
```

This method should coordinate with the provided *IActions* object to execute the following steps in order:

1. Call *unlockTheDoor()* to get the door passcode string.
2. Concurrently call both:
   - *hackSecretPin(passcodeOfDoor)* to get the safe deposit box PIN
   - *figureOutSafeDepositBoxNumber(passcodeOfDoor)* to get the safe deposit box number
3. Call *openSafeDepositLock(safeDepositBoxNumber, safetyDepositBoxPin)* to complete the robbery.

Note: The concurrent operations in step 2 are critical for the time-sensitive nature of the robbery scenario.

Example

The system will validate your implementation by checking if the operations are called in the correct order with the proper parameters. A successful implementation will output:

```
true
```

This indicates that all operations were performed correctly, and the robbery was successful. If the operations are performed in the wrong order or with incorrect parameters, the output will be *false*.

Constraints

- *1* ≤ lengths of *passcodeOfDoor*, *safeDespositBoxNumber, safetyDepositBoxPin* ≤ *10*

▼ INPUT FORMAT FOR CUSTOM TESTING

There is no input passed from stdin.

▼ SAMPLE CASE 0

Sample Output

```
true
```

Explanation

- stdout shows *true* if it is a successful robbery otherwise, it shows *false.*

Interviewer guidelines

Java 8 and Java 15 Solution

```
class SafeCracking {
  public void rob(IActions actions) throws InterruptedException, ExecutionException{
    ExecutorService threadPool = Executors.newFixedThreadPool(2);
```

```
        String passcodeOfDoor = actions.unlockTheDoor();
        Future<String> hackSecretPinTask = threadPool.submit(() -> actions.hackSecretPin(passcodeOf
Door));
        Future<String> figureOutSafeDepositBoxNumberTask = threadPool.submit(() -> actions.figureO
utSafeDepositBoxNumber(passcodeOfDoor));

        String safeDespositBoxNumber = figureOutSafeDepositBoxNumberTask.get();
        String safeDepositBoxPin = hackSecretPinTask.get();

        actions.openSafeDepositLock(safeDespositBoxNumber, safeDepositBoxPin);
        threadPool.shutdown();
    }
}
```

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws
InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor)
throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber,
final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}

import java.io.*;
import java.util.*;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
```

```
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}


class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{


        String passcodeOfDoor = actions.unlockTheDoor();

        CompletableFuture<String> pinFuture = CompletableFuture.supplyAsync(() -> {

            try{
                return actions.hackSecretPin(passcodeOfDoor);
            } catch (Exception e) {
                throw new CompletionException(e);
            }

        });

        CompletableFuture<String> boxNumberFuture = CompletableFuture.supplyAsync(() ->
        {
             try{
                return actions.figureOutSafeDepositBoxNumber(passcodeOfDoor);
             } catch (Exception e){
                throw new CompletionException(e);
             }
        });

        String safetyDepositBoxPin = pinFuture.get();
        String safetyDespositBoxNumber = boxNumberFuture.get();
```

```java

        actions.openSafeDepositLock(safetyDespositBoxNumber, safetyDepositBoxPin);




    }
}
public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
```

```
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

```

```java
    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

```

```
    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	2.1104 sec	30.5 KB
Testcase 1	Hard	Hidden	Success	74	2.1118 sec	30.3 KB

No comments.

3. DB Context

Correct

Coding Medium Context Manager Python Classes OOP

Question description

Implement a class called *database_context_manager* that acts as a context manager for database connections.

The context manager should:

- Create a database connection when entering the context by calling the *_init_db()* method (already defined in the *Database* class)

- Close the connection when exiting the context by calling the *_close_db()* method (also defined in the *Database* class)

Given a string *username* consisting of lowercase English letters, define and complete the *database_context_manager class*.

For a successful simulation:

- The input string username must be passed to *_init_db()* as an argument.
- The connection string returned from *_init_db()* must be passed to *_close_db()* as an argument to close the connection.

Example

Given input *username* = "Sam", if the connection string returned by *_init_db()* is "abc":

- "Sam" must be passed as an argument to *_init_db()*
- "abc" must be passed to *_close_db()*

Function Description

Complete the class *database_context_manager* in the editor with the following parameter(s):

string username: the string to pass to *_init_db()*

Constraints

- $1 \leq |\ username\ | \leq 10^5$
- *username* consists of lowercase English letters.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line and the only line of the test case contains a string, *username*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
Robin
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Robin"*, the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.
- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
Sam
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Sam"*, the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.
- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

Interviewer guidelines

▼ SOLUTION

```
class database_context_manager():

    def __init__(self, username):
        self.username = username
        self.connectionString = ""
        self.obj = Database()
```

```
def __enter__(self):
    self.connectionString = self.obj._init_db(self.username)

def __exit__(self, exc_type, exc_value, exc_traceback):
    self.obj._close_db(self.connectionString)
```

Candidate's Solution

Language used: **Python 3**

```python
#!/bin/python

import math
import os
import random
import re
import sys

fptr = open(os.environ['OUTPUT_PATH'], 'w')

class Database:

    totalCount: int = 0
    count1: int = 0
    count2: int = 0
    inputString: int = ""
    __connString: int = ""

    def _init_db(self, username: str) -> str:

        Database.totalCount += 1
        Database.__connString = ""

        if username == Database.inputString:

            Database.count1 += 1

            le: int = random.randint(1, 100000)
            for x in range(le):
                nxt: int = random.randint(1, 100000)
                nxt %= 26
                Database.__connString += chr(97 + nxt)

            fptr.write('Username matched, database connection initialized\n')

```

```
        return Database.__connString

    def _close_db(self, connectionString: str) -> None:

        Database.totalCount += 1
        if Database.__connString == connectionString:
            Database.count2 += 1
            fptr.write('Connection string matched, database connection closed\n')


#
# Complete the class 'database_context_manager' below.
#
# The class accepts STRING username as parameter.
#

class database_context_manager:

    def __init__(self, username):
        self.username = username

    # write your code here

        self.connection_string = None
        self.db_instance = Database()

    def __enter__(self):
        self.connection_string = self.db_instance._init_db(self.username)
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.db_instance._close_db(self.connection_string)





if __name__ == '__main__':

    username: str = input()
    Database.inputString = username

    with database_context_manager(username) as dbSimulator:
        fptr.write('Database running...\n')

```

```
    tc: int = Database.totalCount
    cnt1: int = Database.count1
    cnt2: int = Database.count2

    if cnt1 != 1 or cnt2 != 1 or tc != 2:
        fptr.write('Functions not correctly called\n')
    fptr.close()

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0566 sec	10.5 KB
TestCase 1	Easy	Sample	Success	1	0.0582 sec	10.6 KB
TestCase 2	Easy	Sample	Success	1	0.1344 sec	10.9 KB
TestCase 3	Easy	Hidden	Success	5	0.2103 sec	11 KB
TestCase 4	Easy	Hidden	Success	5	0.0468 sec	10.8 KB
TestCase 5	Easy	Hidden	Success	6	0.1336 sec	10.9 KB
TestCase 6	Easy	Hidden	Success	6	0.1584 sec	10.9 KB
TestCase 7	Easy	Hidden	Success	10	0.0537 sec	10.9 KB
TestCase 8	Easy	Hidden	Success	10	0.0329 sec	10.9 KB
TestCase 9	Easy	Hidden	Success	10	0.0503 sec	10.9 KB

TestCase 10	Easy	Hidden	Success	10	0.0443 sec	10.9 KB
TestCase 11	Easy	Hidden	Success	10	0.1627 sec	11 KB

No comments.